Assignment 1 - Use Any Public Dataset and Perform Basic Operations on Pandas DataFrame

```
import pandas as pd


# Load public dataset

url = "https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv"

df = pd.read_csv(url)


print("First 3 rows of dataset")

print(df.head(3))


print("\nDataset shape")

print(df.shape)


print("\nColumn names")

print(df.columns)


print("\nDataset information")

print(df.info())


print("\nChecking missing values")
```

```
print(df.isnull().sum())

print("\nStatistical summary")
print(df.describe())

print("\nFilter rows where species is setosa")
print(df[df['species'] == 'setosa'].head(3))

print("\nAverage sepal length")
print(df['sepal_length'].mean())

print("\nSorting by petal length")
print(df.sort_values(by='petal_length', ascending=False).head(3))

print("\nGrouping by species")
print(df.groupby('species').mean())
```

OUTPUT : -

```
First 3 rows of dataset
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa

Dataset shape
(150, 5)

Column names
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='object')

Dataset information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

```
Checking missing values
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64

Statistical summary
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000

Filter rows where species is setosa
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa

Average sepal length
5.843333333333334

Sorting by petal length
```

```
Sorting by petal length
     sepal_length  sepal_width  petal_length  petal_width    species
118           7.7          2.6           6.9          2.3  virginica
117           7.7          3.8           6.7          2.2  virginica
122           7.7          2.8           6.7          2.0  virginica

Grouping by species
            sepal_length  sepal_width  petal_length  petal_width
species
setosa             5.006        3.428         1.462        0.246
versicolor         5.936        2.770         4.260        1.326
virginica          6.588        2.974         5.552        2.026
```

Assignment 2 - Data Visualization Using Python and Pandas

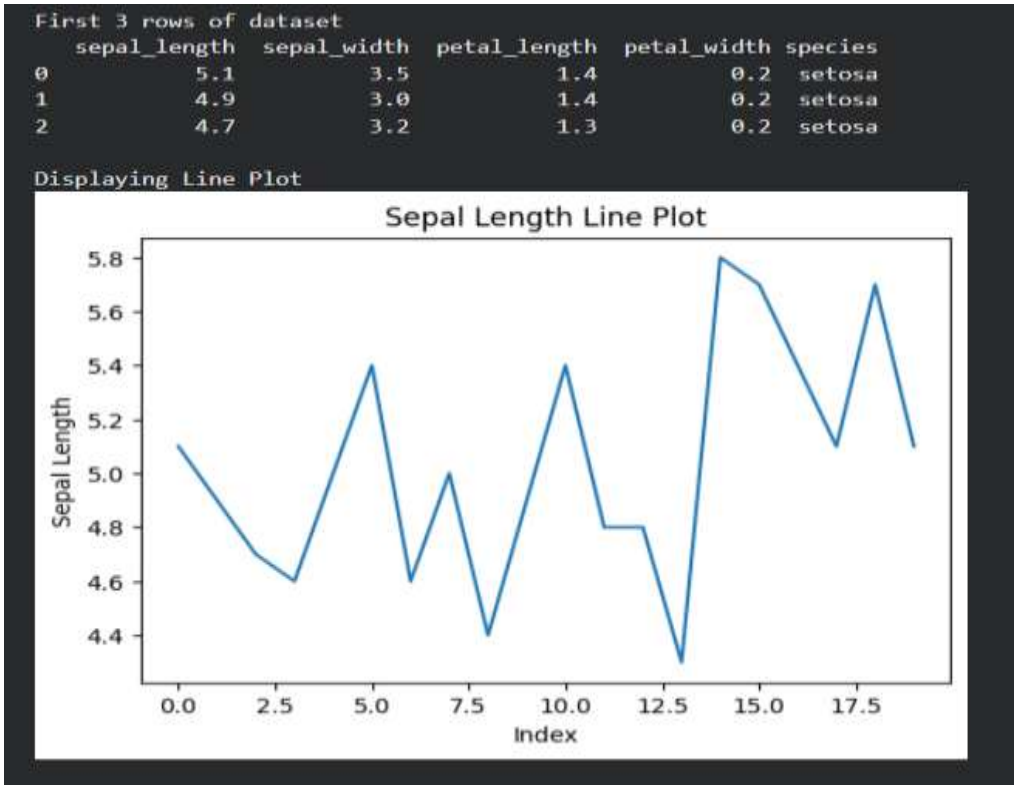
```
import pandas as pd

import matplotlib.pyplot as plt


# Load public dataset

url = "https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv"

df = pd.read_csv(url)


print("First 3 rows of dataset")

print(df.head(3))


# Line plot

print("\nDisplaying Line Plot")

plt.figure(figsize=(6,4))

plt.plot(df['sepal_length'].head(20))

plt.title("Sepal Length Line Plot")

plt.xlabel("Index")

plt.ylabel("Sepal Length")

plt.show()


# Bar plot
```

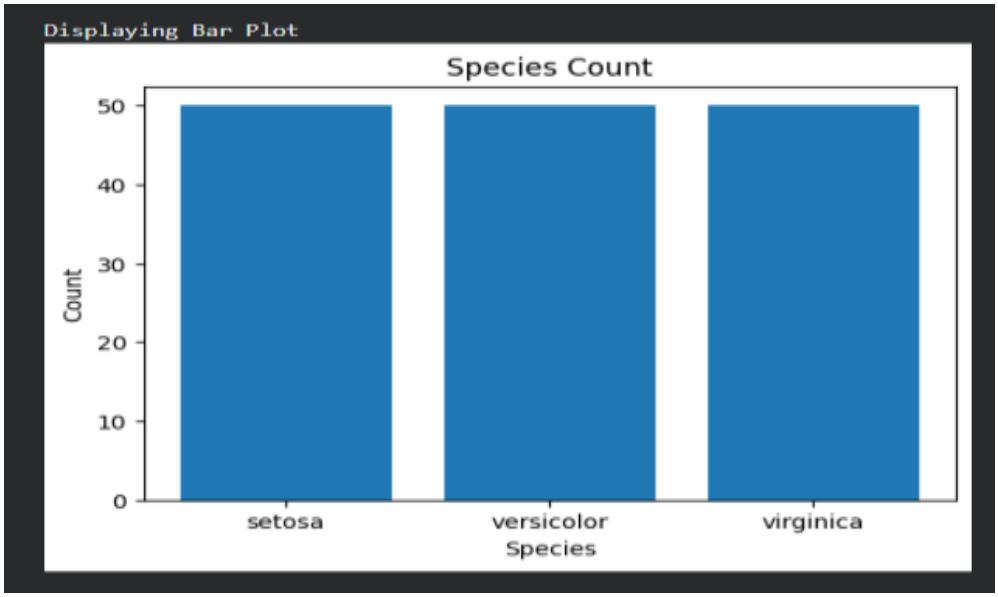
```
print("\nDisplaying Bar Plot")
species_count = df['species'].value_counts()

plt.figure(figsize=(6,4))
plt.bar(species_count.index, species_count.values)
plt.title("Species Count")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()

# Scatter plot
print("\nDisplaying Scatter Plot")
plt.figure(figsize=(6,4))
plt.scatter(df['sepal_length'], df['petal_length'])
plt.title("Sepal Length vs Petal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()

# Histogram
print("\nDisplaying Histogram")
plt.figure(figsize=(6,4))
plt.hist(df['sepal_width'])
plt.title("Sepal Width Distribution")
plt.xlabel("Sepal Width")
```

plt.ylabel("Frequency")

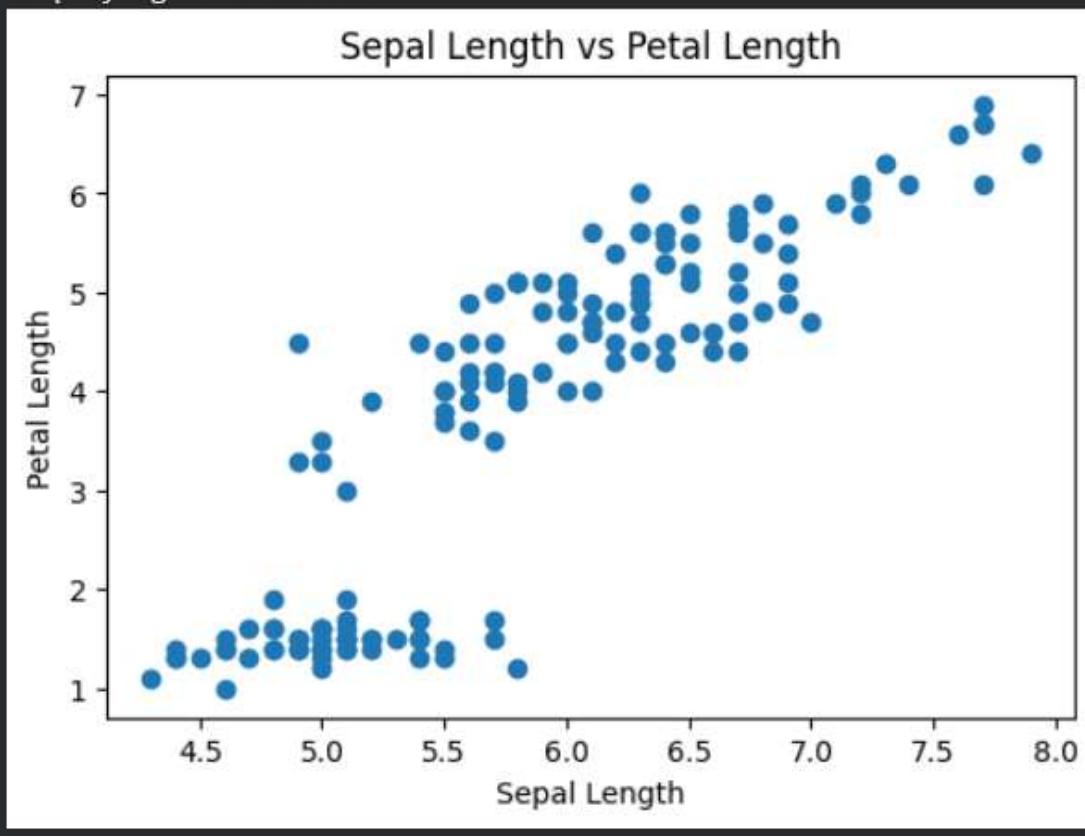
plt.show()

OUTPUT :-

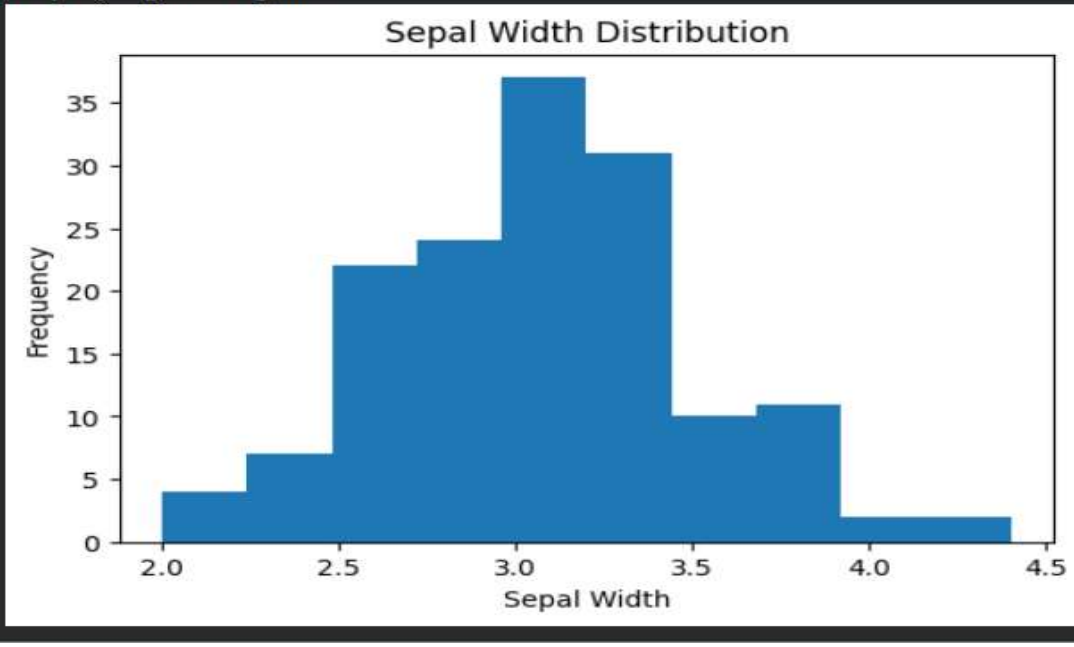




Displaying Scatter Plot



Displaying Histogram



Assignment 3 - Generate Random Numbers and Perform Statistical Analysis with Data Visualization Using Python

```
import random
import numpy as np
import matplotlib.pyplot as plt
from statistics import mean, median, mode

# Generate 100 random numbers between 1 and 100
numbers = [random.randint(1, 100) for i in range(100)]

print("First 10 random numbers")
print(numbers[:10])

# Calculate mean
mean_value = mean(numbers)
print("\nMean of numbers")
print(mean_value)

# Calculate median
median_value = median(numbers)
print("\nMedian of numbers")
print(median_value)
```

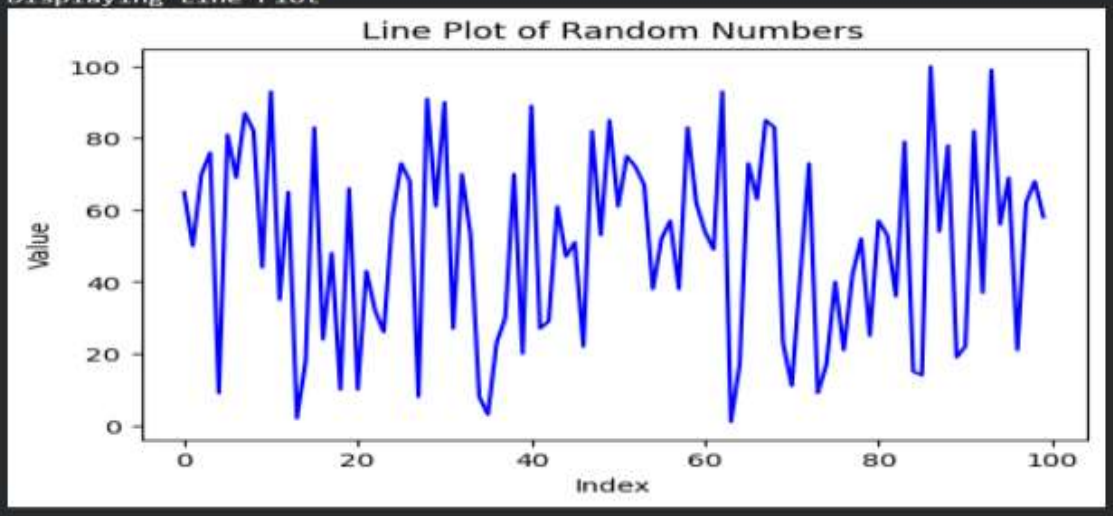
```
# Calculate mode
mode_value = mode(numbers)
print("\nMode of numbers")
print(mode_value)

# Calculate IQR
q1 = np.percentile(numbers, 25)
q3 = np.percentile(numbers, 75)
iqr = q3 - q1

print("\nInter Quartile Range (IQR)")
print(iqr)

# Line Plot
print("\nDisplaying Line Plot")
plt.figure(figsize=(6,4))
plt.plot(numbers, color='blue')
plt.title("Line Plot of Random Numbers")
plt.xlabel("Index")
plt.ylabel("Value")
plt.show()

# Histogram
print("\nDisplaying Histogram")
```

```
plt.figure(figsize=(6,4))
plt.hist(numbers, color='green')
plt.title("Histogram of Random Numbers")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

# Box Plot
print("\nDisplaying Box Plot")
plt.figure(figsize=(6,4))
plt.boxplot(numbers)
plt.title("Box Plot of Random Numbers")
plt.show()

# Scatter Plot
print("\nDisplaying Scatter Plot")
plt.figure(figsize=(6,4))
plt.scatter(range(100), numbers, color='red')
plt.title("Scatter Plot of Random Numbers")
plt.xlabel("Index")
plt.ylabel("Value")
plt.show()
```

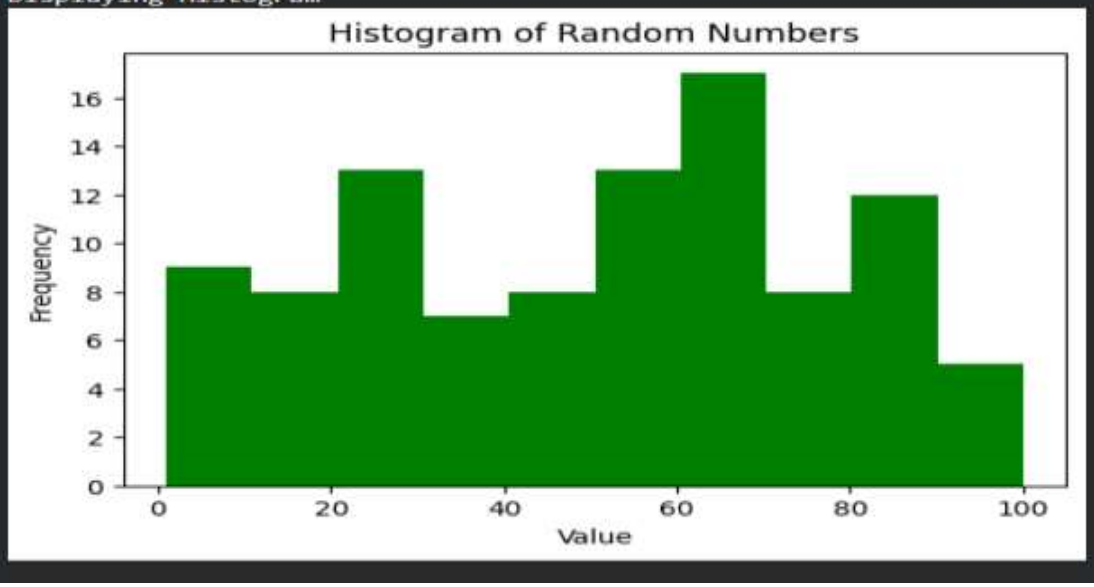
OUTPUT:-

```
First 10 random numbers
[65, 50, 70, 76, 9, 81, 69, 87, 82, 44]

Mean of numbers
50.44

Median of numbers
53.0

Mode of numbers
70

Inter Quartile Range (IQR)
44.75
```

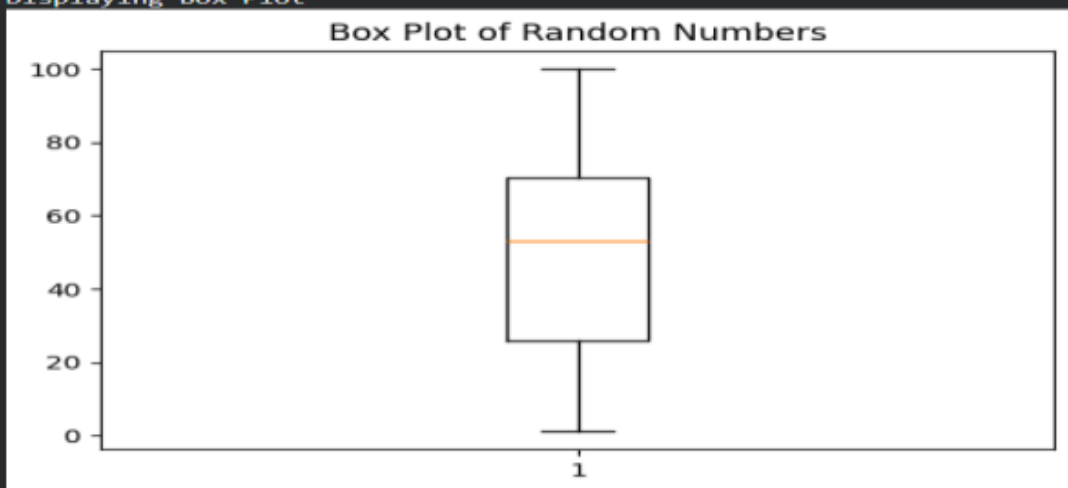
Displaying Line Plot



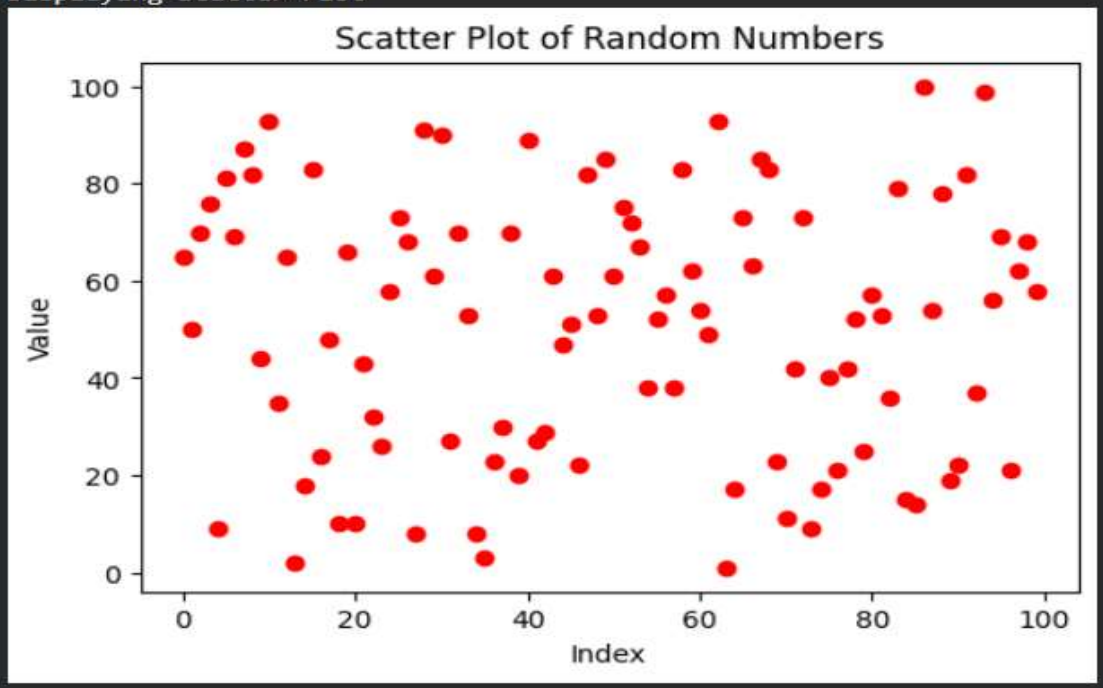
Displaying Histogram



Displaying Box Plot



Displaying Scatter Plot



Assignment 4 - Pearson’s Correlation Coefficient and Scatter Matrix Plot Using Iris Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add class column
df['class'] = iris.target

print("First 3 rows of dataset")
print(df.head(3))

# Select three feature pairs
print("\nPearson Correlation Coefficient for Feature Pairs")
```

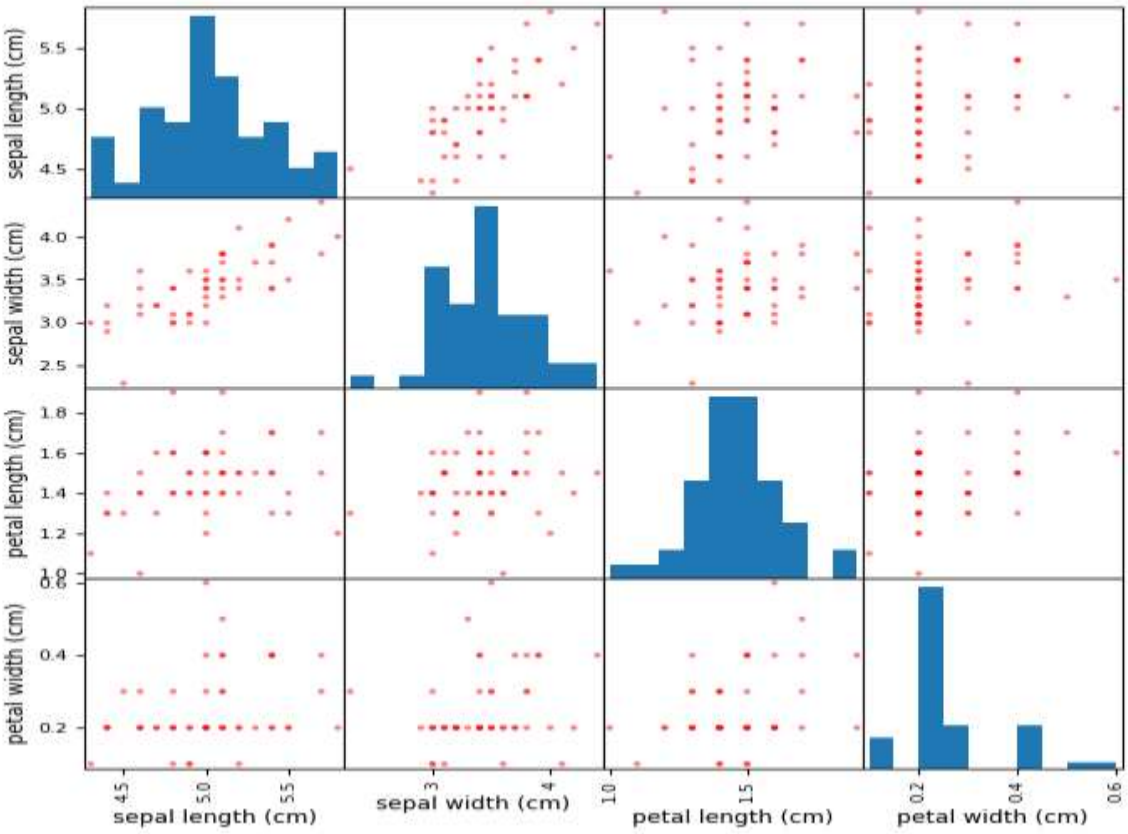
```
corr1 = df['sepal length (cm)'].corr(df['sepal width (cm)'])
print("\nSepal Length vs Sepal Width")
print(corr1)

corr2 = df['petal length (cm)'].corr(df['petal width (cm)'])
print("\nPetal Length vs Petal Width")
print(corr2)

corr3 = df['sepal length (cm)'].corr(df['petal length (cm)'])
print("\nSepal Length vs Petal Length")
print(corr3)

# Compute PCC between features and class
print("\nPearson Correlation Coefficient between Features and Class")

for column in df.columns[:-1]:
    value = df[column].corr(df['class'])
    print(column, ":", value)

# Scatter Matrix Plot for each class separately

# Setosa
print("\nDisplaying Scatter Matrix for Setosa")
setosa = df[df['class'] == 0]
```

```
scatter_matrix(setosa.iloc[:, :-1], figsize=(8,8), color='red')
plt.suptitle("Scatter Matrix - Setosa")
plt.show()

# Versicolor
print("\nDisplaying Scatter Matrix for Versicolor")
versicolor = df[df['class'] == 1]

scatter_matrix(versicolor.iloc[:, :-1], figsize=(8,8), color='green')
plt.suptitle("Scatter Matrix - Versicolor")
plt.show()

# Virginica
print("\nDisplaying Scatter Matrix for Virginica")
virginica = df[df['class'] == 2]

scatter_matrix(virginica.iloc[:, :-1], figsize=(8,8), color='blue')
plt.suptitle("Scatter Matrix - Virginica")
plt.show()
```

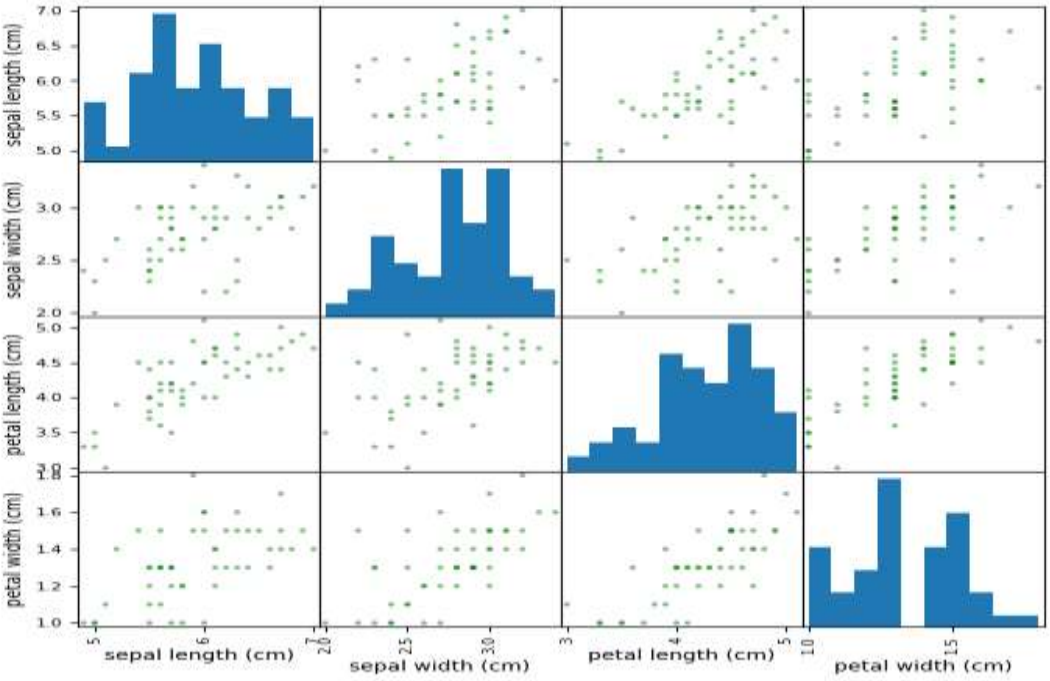
OUTPUT:-

```
First 3 rows of dataset
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2

   class
0      0
1      0
2      0

Pearson Correlation Coefficient for Feature Pairs

Sepal Length vs Sepal Width
-0.11756978413300208

Petal Length vs Petal Width
0.9628654314027961

Sepal Length vs Petal Length
0.8717537758865831

Pearson Correlation Coefficient between Features and Class
sepal length (cm) : 0.782561231810082
sepal width (cm) : -0.4266575607811244
petal length (cm) : 0.9490346990083889
petal width (cm) : 0.9565473328764034
```

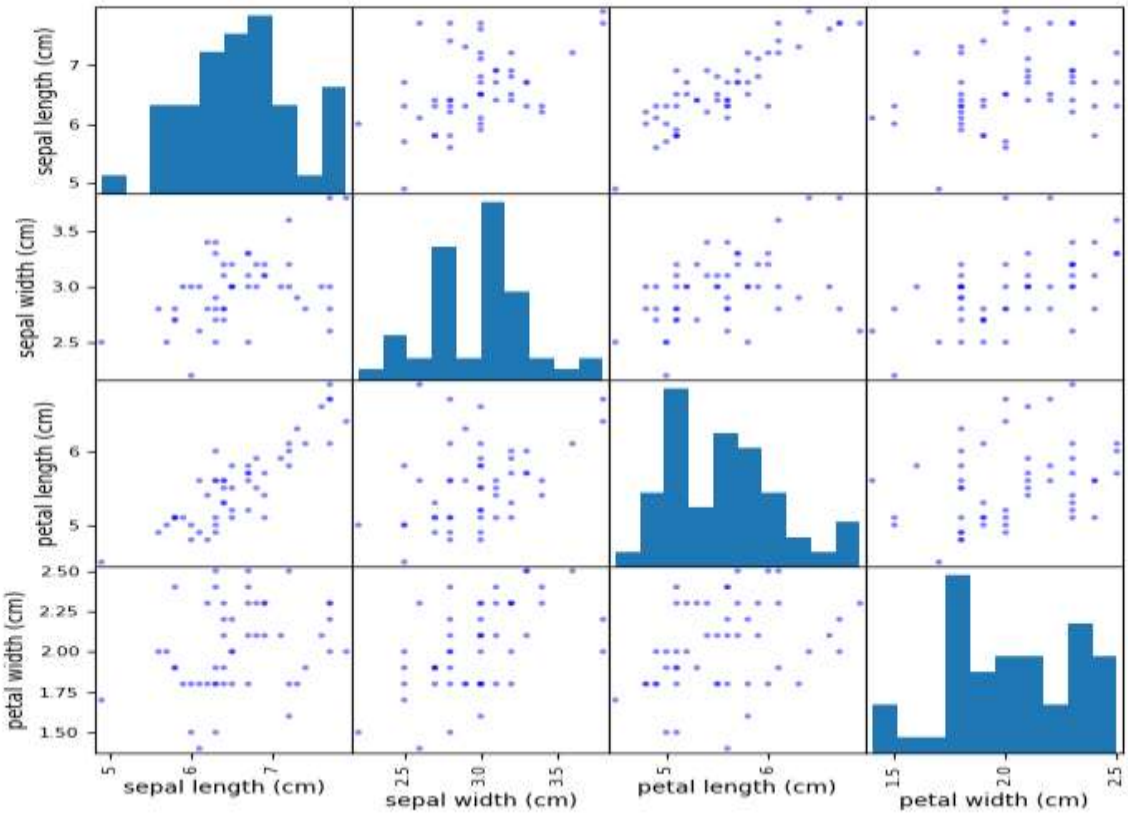
Scatter Matrix - Setosa



Scatter Matrix - Versicolor



Scatter Matrix - Virginica



Assignment 5 - Spearman Correlation Coefficient Using Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add class column
df['class'] = iris.target

print("First 3 rows of dataset")
print(df.head(3))

# Spearman Correlation between feature pairs
print("\nSpearman Correlation Coefficient for Feature Pairs")

corr1 = df['sepal length (cm)'].corr(df['sepal width (cm)'], method='spearman')
print("\nSepal Length vs Sepal Width")
print(corr1)
```

```
corr2 = df['petal length (cm)'].corr(df['petal width (cm)'], method='spearman')
print("\nPetal Length vs Petal Width")
print(corr2)

corr3 = df['sepal length (cm)'].corr(df['petal length (cm)'], method='spearman')
print("\nSepal Length vs Petal Length")
print(corr3)

# Spearman Correlation between features and class
print("\nSpearman Correlation between Features and Class")

for column in df.columns[:-1]:
    value = df[column].corr(df['class'], method='spearman')
    print(column, ":", value)

# Complete Spearman Correlation Matrix
print("\nComplete Spearman Correlation Matrix")
print(df.corr(method='spearman'))
```

OUTPUT:-

```
First 3 rows of dataset
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2

   class
0      0
1      0
2      0

Spearman Correlation Coefficient for Feature Pairs

Sepal Length vs Sepal Width
-0.166777658283235

Petal Length vs Petal Width
0.9376668235763412

Sepal Length vs Petal Length
0.881898126434986

Spearman Correlation between Features and Class
sepal length (cm) : 0.7980781172420549
sepal width (cm) : -0.440289584131434
petal length (cm) : 0.9354305169914044
petal width (cm) : 0.9381791663400608

Complete Spearman Correlation Matrix
                   sepal length (cm)  sepal width (cm)  petal length (cm)  \
sepal length (cm)           1.000000         -0.166778           0.881898
sepal width (cm)           -0.166778          1.000000          -0.309635
petal length (cm)           0.881898         -0.309635           1.000000
petal width (cm)            0.834289         -0.289032           0.937667
class                       0.798078         -0.440290           0.935431

                   petal width (cm)     class
sepal length (cm)          0.834289  0.798078
sepal width (cm)          -0.289032 -0.440290
petal length (cm)          0.937667  0.935431
petal width (cm)           1.000000  0.938179
class                      0.938179  1.000000
```

Assignment 6 - Simple Linear Regression on Real Estate Price Prediction

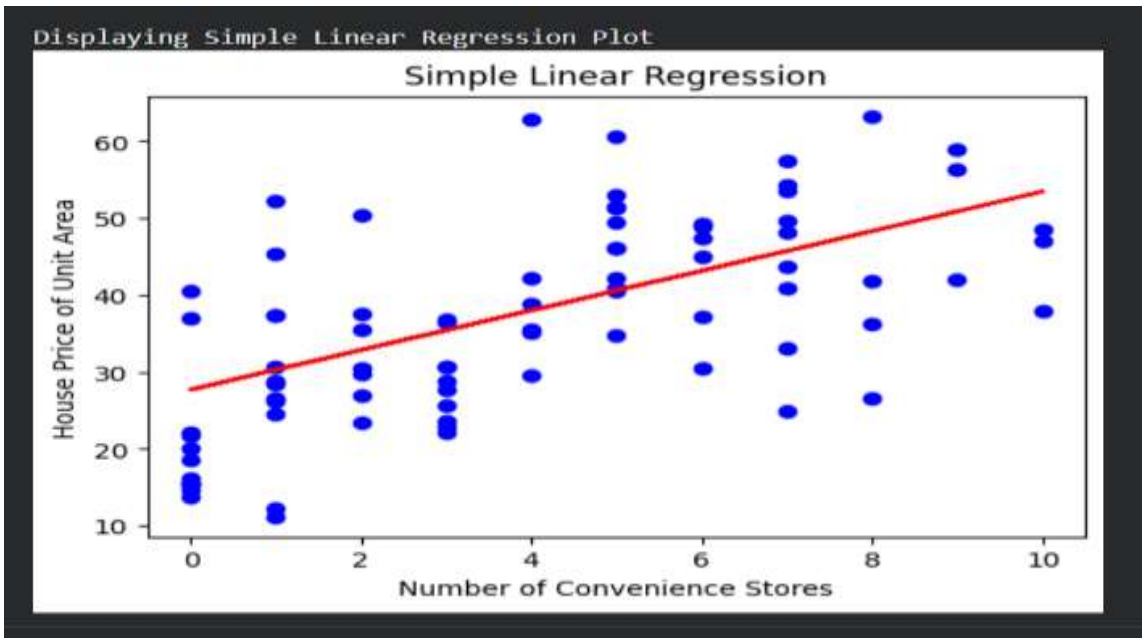
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("Real estate.csv")

print("First 3 rows of dataset")
print(df.head(3))

# Select single feature and target
X = df[['X4 number of convenience stores']]
y = df['Y house price of unit area']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)

print("\nModel Coefficient")
print(model.coef_)

print("\nModel Intercept")
print(model.intercept_)

print("\nMean Squared Error")
print(mean_squared_error(y_test, y_pred))

print("\nR2 Score")
print(r2_score(y_test, y_pred))

# Plot regression line
print("\nDisplaying Simple Linear Regression Plot")
plt.figure(figsize=(6,4))
plt.scatter(X_test, y_test, color='blue')
```

```
plt.plot(X_test, y_pred, color='red')

plt.title("Simple Linear Regression")

plt.xlabel("Number of Convenience Stores")

plt.ylabel("House Price of Unit Area")

plt.show()
```

OUTPUT : -

```
First 3 rows of dataset
   No  X1 transaction date  X2 house age  \
0   1              2012.917          32.0
1   2              2012.917          19.5
2   3              2013.583          13.3

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5

   X5 latitude  X6 longitude  Y house price of unit area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3

Model Coefficient
[2.57738965]

Model Intercept
27.70822173392014

Mean Squared Error
101.73839399811925

R2 Score
0.3935470378244481

Displaying Simple Linear Regression Plot
```



Assignment 7 - Multiple Linear Regression on Real Estate Price Prediction

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("Real estate.csv")
print("First 3 rows of dataset")
print(df.head(3))
# Select multiple features
X = df[
    [
        'X2 house age',
        'X3 distance to the nearest MRT station',
        'X4 number of convenience stores'
    ]
]
# Target variable
y = df['Y house price of unit area']
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)
# Predict values
y_pred = model.predict(X_test)
print("\nModel Coefficients")
print(model.coef_)
print("\nModel Intercept")
print(model.intercept_)
print("\nMean Squared Error")
print(mean_squared_error(y_test, y_pred))
print("\nR2 Score")
print(r2_score(y_test, y_pred))
# Actual vs Predicted Plot
print("\nDisplaying Actual vs Predicted Plot")

plt.figure(figsize=(6,4))
plt.scatter(y_test, y_pred, color='green')

plt.title("Multiple Linear Regression")
```

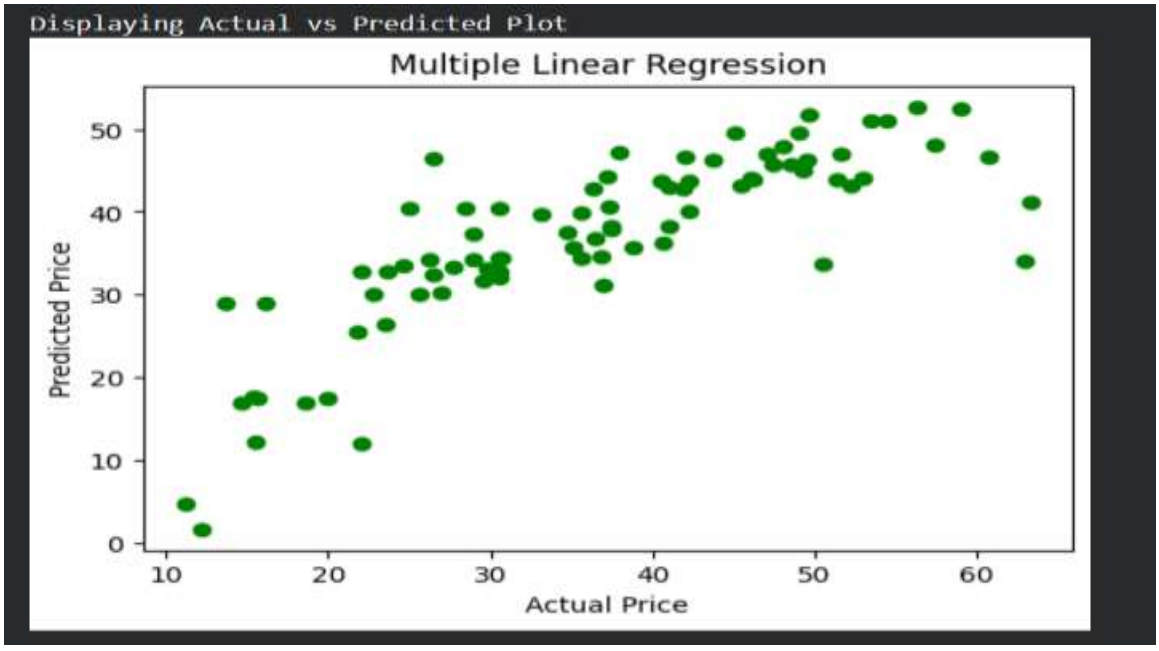
plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()

OUTPUT : -

```
First 3 rows of dataset
   No  X1 transaction date  X2 house age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5

   X5 latitude  X6 longitude  Y house price of unit area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3

Model Coefficients
[-0.25840543 -0.00549834  1.24734248]

Model Intercept
43.5148801440439

Mean Squared Error
58.88825128983575

R2 Score
0.6489726933106557
```



Assignment 8 - Non Linear Regression on Real Estate Price Prediction Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("Real estate.csv")
print("First 3 rows of dataset")
print(df.head(3))
# Select feature and target
X = df[['X4 number of convenience stores']]
y = df['Y house price of unit area']
# Convert into polynomial features
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_poly, y, test_size=0.2, random_state=42
```

```
)

# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)
print("\nModel Coefficients")
print(model.coef_)
print("\nModel Intercept")
print(model.intercept_)
print("\nMean Squared Error")
print(mean_squared_error(y_test, y_pred))
print("\nR2 Score")
print(r2_score(y_test, y_pred))
# Plot Non Linear Regression
print("\nDisplaying Non Linear Regression Plot")
plt.figure(figsize=(6,4))
plt.scatter(X.iloc[:,0], y, color='blue')
sorted_zip = sorted(zip(X.iloc[:,0], model.predict(X_poly)))
x_poly, y_poly = zip(*sorted_zip)
plt.plot(x_poly, y_poly, color='red')
plt.title("Non Linear Regression")
```

plt.xlabel("Number of Convenience Stores")

plt.ylabel("House Price of Unit Area")

plt.show()

OUTPUT : -

```
First 3 rows of dataset
   No  X1 transaction date  X2 house age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                               84.87882                               10
1                              306.59470                                9
2                              561.98450                                5

   X5 latitude  X6 longitude  Y house price of unit area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3

Model Coefficients
[ 0.          3.04185425 -0.05385946]

Model Intercept
27.17684610562167

Mean Squared Error
99.66433277272472

R2 Score
0.40591032099065594

Displaying Non Linear Regression Plot
```